

3 отчёт по внешнему курсу

Выполнение работы

Цель работы

Научиться работе в ос Linux на базовом уровне.

Задание

Выполнить курс на stepic. 2 этап

Теоретическое введение

Курс выполнялся на платформе stepic.

Выполнение лабораторной работы

Выполнение 1 задания (рис. 1).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✓ Правильно, молодец!

- ☐ "Esc"
- ☐ "Ctrl", затем "x"
- ☐ ":", затем "q"
- ☒ ":", затем "q", затем "Enter"
- ☐ "Q"

[Следующий шаг](#) [Решить снова](#)

Задание 1

Выполнение 2 задания (рис. 2).

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ В этой строке 5 "больших слов" (WORD)
- ☐ В этой строке 12 "слов" (word)
- ☒ В этой строке 9 "слов" (word)
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

Следующий шаг

Решить снова

Задание 2

Выполнение 3 задания (рис. 3).

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже **наборов нажатий клавиш** выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ x2wwywp
- ☒ d2wwywp
- ☒ ddithree four four four five<Esc>
- ☒ d2wwifour four <Esc>
- ☐ d2dwywГр
- ☐ d2wwywp

Следующий шаг

Решить снова

Задание 3

Выполнение 4 задания (рис. 4).

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Верно. Так держать!

```
:%s/Windows/Linux
```

Следующий шаг

Решить снова

Задание 4

Выполнение 5 задания (рис. 5).

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ В режиме выделения можно использовать команды d (удалить) и u (скопировать)
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)

Следующий шаг

Решить снова

Задание 5

Выполнение 6 задания (рис. 6).

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ Никакие команды появляться не будут
- ☒ Только из набора C
- ☐ Из наборов A и C
- ☐ Только из набора A
- ☐ Из наборов B и C

Следующий шаг

Решить снова

Задание 6

Выполнение 7 задания (рис. 7).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Правильно.

- ☐ `/home/bi/Desktop/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`
- ☒ `/home/bi/file1.txt`
- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)

Следующий шаг

Решить снова

Задание 7

Выполнение 8 задания (рис. 8).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ variable
- ☐ 123variable
- ☐ var@iable
- ☒ _variable
- ☒ __variable
- ☒ variable123
- ☒ VARiable

Следующий шаг

Решить снова

Задание 8

Выполнение 9 задания (рис. 9).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 var1=$1
3 var2=$2
4
5 echo "Arguments are: \ $1=$var1 \ $2=$var2"
6
7
8
```

Следующий шаг

Решить снова

Задание 9

Выполнение 10 задания (рис. 10).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `0 -ge 0`
- ☒ `-s $0`
- ☒ `-n $0`
- ☐ `0 -gt 0`
- ☒ `5 -ge 5`
- ☐ `-z ""`

Следующий шаг

Решить снова

Задание 10

Выполнение 11 задания (рис. 11).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Верно.

- ☐ Сначала two, потом four
- ☐ Сначала four, потом one
- ☐ Сначала one, потом two
- ☒ Сначала four, потом four

Следующий шаг

Решить снова

Задание 11

Выполнение 12 задания (рис. 12).

Напишите скрипт на `bash`, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверять вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`

✓ Хорошие новости, верно!

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2   echo "1 student"
3 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
4   echo "$1 students"
5 elif [[ $1 -ge 5 ]]; then
6   echo "A lot of students"
7 else
8   echo "No students"
9 fi
10
11
12
13
14
```

Следующий шаг

Решить снова

Задание 12

Выполнение 13 задания (рис. 13).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент `bash`-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Выберите один вариант из списка

✓ Правильно, молодец!

- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☐ 5 раз "start" и 2 раза "finish"

Следующий шаг

Решить снова

Задание 13

Выполнение 14 задания (рис. 14).

Напишите скрипт на Bash, который будет определять в какую возрастную группу попадает пользователь. При запуске скрипта должен выводиться сообщение "enter your name" и ждать от пользователя ввод имени (переменная read), чтобы прочитать его). Когда имя введено, то скрипт должен написать "enter your age" и ждать ввода возраста (переменная read). Когда возраст введён, скрипт пишет на экран "Okname, your group is child", где group определяется на основе возраста по следующим правилам:

- меньше либо равно 16: 'child',
- от 17 до 25 (включительно): 'youth',
- старше 25: 'adult'.

После этого скрипт опять выводит сообщение "enter your name" и всё начинается по новой (бесконечный цикл). Если в какой-то момент работы скрипта будет введено пустое имя или возраст 0, то скрипт должен написать на экран "bye" и закончить свою работу (выход из цикла).

Примеры корректной работы скрипта:

```
181
./script.sh
enter your name:
Ege
enter your age:
16
Ege, your group is child
enter your name:
Olga
enter your age:
0
bye

182
./script.sh
enter your name:
Olga Petrovna
enter your age:
26
Olga Petrovna, your group is youth
enter your name:
bye
```

Подсказка: в случае проблем с решением задания, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✔ **Правильно, молодец!**

Теперь вы можете [сверить решение](#), где вы можете сравнить ваше решение с другим или отправить ответ.

Вы можете отправить ответ, подтвердив: вы можете принять предложенное решение в [комментарии](#), ответив на их вопросы, или сравнить свой [решения с другим на странице решения](#).

```
1 #!/usr/bin/bash
2 echo
3
4 while [[ $x != 1 ]]
5 do
6
7 echo "enter your name:"
8 read name
9 if [[ -x $name ]]
10 then echo "bye"
11
12     exit
13 fi
14 echo "enter your age:"
15 read age
16 if [[ $age -eq 0 ]]
17 then echo "bye"
18
19     exit
20 elif [[ $age -le 16 ]]
21 then echo "$name, your group is child"
22 elif [[ $age -le 25 || $age -ge 17 ]]
23 then echo "$name, your group is youth"
24 else echo "$name, your group is adult"
25
26 fi
```

Следующий шаг

Решить снова

Задание 14

Выполнение 15 задания (рис. 15).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите все подходящие варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `let "a = a + b"`
- ☒ `let "a=${a+$b}"`
- ☒ `let "a+=b"`
- ☒ `let a=a+b`
- ☐ `a=${a+$b}`

Следующий шаг

Решить снова

Задание 15

Выполнение 16 задания (рис. 16).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash

cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ ``pwd``
- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☐ `/home/bi/Documents`
- ☐ `pwd`
- ☒ `/home/bi`

Следующий шаг

Решить снова

Задание 16

Выполнение 17 задания (рис. 17).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (```), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `if `program > some_file.txt``
- ☐ `if [[ `program` -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☐ Сначала `var=`program``, затем `if [[ $var -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`

Следующий шаг

Решить снова

Задание 17

Выполнение 18 задания (рис. 18).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из `bash`-скрипта:

```
counter () # takes one argument
{
    local let "c1+=1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже строку, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте после десяти вызовов функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на `bash`, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Задание 18

Выполнение 19 задания (рис. 19).

Напишите скрипт на `bash`, который будет искать наибольший общий делитель (`НОД`, `greatest common divisor`, `GCD`) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто выводить результат работы скрипта через `printf` (для этого можно использовать `%d %d`) и указывать ему два параметра — `pi`, `pr` (пример в видеоброфайле). После ввода чисел скрипт считывает `НОД` и выводит на экран сообщение "`GCD` `ix` `кратчайшее значение`", например для чисел 15 и 25 это будет "`GCD` `ix` 5". После этого скрипт опять выводит в экран сообщение двух натуральных чисел. Если в какой-то момент работы пользоваться `exit` вместо этого лучше не стоит, то можно написать на экран "`bye`" и закончить свою работу.

Выполнение `НОД` можно реализовать с помощью алгоритма Евклида. Если кратко написать функцию `gcd`, которая принимает на вход два аргумента (буквенно `M` и `N`). Если аргументы равны, то мы нашли `НОД` — он равен `M` (или `N`), можно вывести соответствующее сообщение на экран (или `exit`). Иначе можно сравнить аргументы между собой. Если `M` больше `N`, то вызовем ту же функцию `gcd`, но в качестве первого аргумента передадим `(M-N)`, а в качестве второго `N`. Если же наоборот, `M` меньше `N`, то вызовем функцию `gcd` с первым аргументом `M`, а вторым `(N-M)`.

Пример корректной работы скрипта:

```
./script.sh
15 25
GCD ix 5
7 5
GCD ix 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.е. можно вызывать `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте можно использовать инструкцию `return` (как инструкции функции после `return` выполниться не будет). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в редкой версии можно обойтись и без использования `return`.

Подсказка: в случае проблемы с решением задачи, обратит внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через `stdin → stdout`

✓ Тестовый
 Тесты выдают пример (можно увидеть), где вы можете сравнить свое решение с другим или отправить свое.

```
1 while { true }
2 do
3   read x2 x1
4   if [ -n "$x2" ]; then
5     echo "bye"
6     break
7   fi
8   gcd () {
9     gcdlong=1
10    if [ $x2 -ne 0 ];
11    then
12      echo "bye"
13      fi
14    while [ $gcdlong -ne 0 ]
15    do
16      gcdlong=$((x1%$x2))
17    done
18    x1=$gcdlong
19    x2=$((x2-$gcdlong))
20  }
21  gcd $x2 $x1
22  echo "GCD ix $x1"
23 fi
24 done
25
26
27
28
```

Задание 19

Выполнение 20 задания (рис. 20).

Напишите калькулятор на Bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом не экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово "exit". В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. Три аргумента через пробел: первый операнд (целое число), оператор (одно из "+", "-", "*", "/", "%", "^") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого вывести в экран слово "bye" и завершить работу.
3. Любая другая команда на одного аргументе или на трех аргументах, но с оператором на их месте. В этом случае нужно вывести на экран слово "error" и завершить работу.

Чтобы проверить работу скрипта, вы можете вызвать сразу несколько команд в файле и передать его скрипту на вход (т.е. выполнить `./calculator.sh < script.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 5
2 * 10
exit
```

то на экране будет:

```
12
20
bye
```

Если же не ввод поступит следующий файл:

```
5 - 5
2/10
exit
```

то на экране будет:

```
error
```

т.е. вторая команда была неверной (в ней всего один аргумент, т.е. нет пробела между числом и оператором), а единственная правильная команда на одного аргумента это "exit".

Подсказка: в случае проблемы с решением задан, обратитесь внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через stdin → stdout

✔ Правильно.

Поскольку вы получили **полное решение**, то вы можете отправить свое решение с другим или отложить совет.

```
1 #!/bin/bash
2 while [[ $1 != "" ]]
3 do
4     read -r result
5     if [[ $result == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ $result =~ ^"([0-9]+)"$ ]] || [[ $result =~ ^"([0-9]+)"$ ]]
10    then
11        echo "error"
12        break
13    else
14        read -r op
15        if [[ $op == "+" ]]
16        then
17            result=$((result + $2))
18        elif [[ $op == "-" ]]
19        then
20            result=$((result - $2))
21        elif [[ $op == "*" ]]
22        then
23            result=$((result * $2))
24        elif [[ $op == "/" ]]
25        then
26            result=$((result / $2))
27        elif [[ $op == "%" ]]
28        then
29            result=$((result % $2))
30        elif [[ $op == "^" ]]
31        then
32            result=$((result ^ $2))
33        else
34            echo "error"
35        fi
36        echo $result
37    fi
38    read -r op
39    read -r op2
40    read -r op3
41    read -r op4
42    read -r op5
43    read -r op6
44    read -r op7
45    read -r op8
46    read -r op9
47    read -r op10
48    read -r op11
49    read -r op12
50    read -r op13
51    read -r op14
52    read -r op15
53    read -r op16
54    read -r op17
55    read -r op18
56    read -r op19
57    read -r op20
58    read -r op21
59    read -r op22
60    read -r op23
61    read -r op24
62    read -r op25
63    read -r op26
64    read -r op27
65    read -r op28
66    read -r op29
67    read -r op30
68    read -r op31
69    read -r op32
70    read -r op33
71    read -r op34
72    read -r op35
73    read -r op36
74    read -r op37
75    read -r op38
76    read -r op39
77    read -r op40
78    read -r op41
79    read -r op42
80    read -r op43
81    read -r op44
82    read -r op45
83    read -r op46
84    read -r op47
85    read -r op48
86    read -r op49
87    read -r op50
88    read -r op51
89    read -r op52
90    read -r op53
91    read -r op54
92    read -r op55
93    read -r op56
94    read -r op57
95    read -r op58
96    read -r op59
97    read -r op60
98    read -r op61
99    read -r op62
100   read -r op63
101   read -r op64
102   read -r op65
103   read -r op66
104   read -r op67
105   read -r op68
106   read -r op69
107   read -r op70
108   read -r op71
109   read -r op72
110   read -r op73
111   read -r op74
112   read -r op75
113   read -r op76
114   read -r op77
115   read -r op78
116   read -r op79
117   read -r op80
118   read -r op81
119   read -r op82
120   read -r op83
121   read -r op84
122   read -r op85
123   read -r op86
124   read -r op87
125   read -r op88
126   read -r op89
127   read -r op90
128   read -r op91
129   read -r op92
130   read -r op93
131   read -r op94
132   read -r op95
133   read -r op96
134   read -r op97
135   read -r op98
136   read -r op99
137   read -r op100
138   read -r op101
139   read -r op102
140   read -r op103
141   read -r op104
142   read -r op105
143   read -r op106
144   read -r op107
145   read -r op108
146   read -r op109
147   read -r op110
148   read -r op111
149   read -r op112
150   read -r op113
151   read -r op114
152   read -r op115
153   read -r op116
154   read -r op117
155   read -r op118
156   read -r op119
157   read -r op120
158   read -r op121
159   read -r op122
160   read -r op123
161   read -r op124
162   read -r op125
163   read -r op126
164   read -r op127
165   read -r op128
166   read -r op129
167   read -r op130
168   read -r op131
169   read -r op132
170   read -r op133
171   read -r op134
172   read -r op135
173   read -r op136
174   read -r op137
175   read -r op138
176   read -r op139
177   read -r op140
178   read -r op141
179   read -r op142
180   read -r op143
181   read -r op144
182   read -r op145
183   read -r op146
184   read -r op147
185   read -r op148
186   read -r op149
187   read -r op150
188   read -r op151
189   read -r op152
190   read -r op153
191   read -r op154
192   read -r op155
193   read -r op156
194   read -r op157
195   read -r op158
196   read -r op159
197   read -r op160
198   read -r op161
199   read -r op162
200   read -r op163
201   read -r op164
202   read -r op165
203   read -r op166
204   read -r op167
205   read -r op168
206   read -r op169
207   read -r op170
208   read -r op171
209   read -r op172
210   read -r op173
211   read -r op174
212   read -r op175
213   read -r op176
214   read -r op177
215   read -r op178
216   read -r op179
217   read -r op180
218   read -r op181
219   read -r op182
220   read -r op183
221   read -r op184
222   read -r op185
223   read -r op186
224   read -r op187
225   read -r op188
226   read -r op189
227   read -r op190
228   read -r op191
229   read -r op192
230   read -r op193
231   read -r op194
232   read -r op195
233   read -r op196
234   read -r op197
235   read -r op198
236   read -r op199
237   read -r op200
238   read -r op201
239   read -r op202
240   read -r op203
241   read -r op204
242   read -r op205
243   read -r op206
244   read -r op207
245   read -r op208
246   read -r op209
247   read -r op210
248   read -r op211
249   read -r op212
250   read -r op213
251   read -r op214
252   read -r op215
253   read -r op216
254   read -r op217
255   read -r op218
256   read -r op219
257   read -r op220
258   read -r op221
259   read -r op222
260   read -r op223
261   read -r op224
262   read -r op225
263   read -r op226
264   read -r op227
265   read -r op228
266   read -r op229
267   read -r op230
268   read -r op231
269   read -r op232
270   read -r op233
271   read -r op234
272   read -r op235
273   read -r op236
274   read -r op237
275   read -r op238
276   read -r op239
277   read -r op240
278   read -r op241
279   read -r op242
280   read -r op243
281   read -r op244
282   read -r op245
283   read -r op246
284   read -r op247
285   read -r op248
286   read -r op249
287   read -r op250
288   read -r op251
289   read -r op252
290   read -r op253
291   read -r op254
292   read -r op255
293   read -r op256
294   read -r op257
295   read -r op258
296   read -r op259
297   read -r op260
298   read -r op261
299   read -r op262
300   read -r op263
301   read -r op264
302   read -r op265
303   read -r op266
304   read -r op267
305   read -r op268
306   read -r op269
307   read -r op270
308   read -r op271
309   read -r op272
310   read -r op273
311   read -r op274
312   read -r op275
313   read -r op276
314   read -r op277
315   read -r op278
316   read -r op279
317   read -r op280
318   read -r op281
319   read -r op282
320   read -r op283
321   read -r op284
322   read -r op285
323   read -r op286
324   read -r op287
325   read -r op288
326   read -r op289
327   read -r op290
328   read -r op291
329   read -r op292
330   read -r op293
331   read -r op294
332   read -r op295
333   read -r op296
334   read -r op297
335   read -r op298
336   read -r op299
337   read -r op300
338   read -r op301
339   read -r op302
340   read -r op303
341   read -r op304
342   read -r op305
343   read -r op306
344   read -r op307
345   read -r op308
346   read -r op309
347   read -r op310
348   read -r op311
349   read -r op312
350   read -r op313
351   read -r op314
352   read -r op315
353   read -r op316
354   read -r op317
355   read -r op318
356   read -r op319
357   read -r op320
358   read -r op321
359   read -r op322
360   read -r op323
361   read -r op324
362   read -r op325
363   read -r op326
364   read -r op327
365   read -r op328
366   read -r op329
367   read -r op330
368   read -r op331
369   read -r op332
370   read -r op333
371   read -r op334
372   read -r op335
373   read -r op336
374   read -r op337
375   read -r op338
376   read -r op339
377   read -r op340
378   read -r op341
379   read -r op342
380   read -r op343
381   read -r op344
382   read -r op345
383   read -r op346
384   read -r op347
385   read -r op348
386   read -r op349
387   read -r op350
388   read -r op351
389   read -r op352
390   read -r op353
391   read -r op354
392   read -r op355
393   read -r op356
394   read -r op357
395   read -r op358
396   read -r op359
397   read -r op360
398   read -r op361
399   read -r op362
400   read -r op363
401   read -r op364
402   read -r op365
403   read -r op366
404   read -r op367
405   read -r op368
406   read -r op369
407   read -r op370
408   read -r op371
409   read -r op372
410   read -r op373
411   read -r op374
412   read -r op375
413   read -r op376
414   read -r op377
415   read -r op378
416   read -r op379
417   read -r op380
418   read -r op381
419   read -r op382
420   read -r op383
421   read -r op384
422   read -r op385
423   read -r op386
424   read -r op387
425   read -r op388
426   read -r op389
427   read -r op390
428   read -r op391
429   read -r op392
430   read -r op393
431   read -r op394
432   read -r op395
433   read -r op396
434   read -r op397
435   read -r op398
436   read -r op399
437   read -r op400
438   read -r op401
439   read -r op402
440   read -r op403
441   read -r op404
442   read -r op405
443   read -r op406
444   read -r op407
445   read -r op408
446   read -r op409
447   read -r op410
448   read -r op411
449   read -r op412
450   read -r op413
451   read -r op414
452   read -r op415
453   read -r op416
454   read -r op417
455   read -r op418
456   read -r op419
457   read -r op420
458   read -r op421
459   read -r op422
460   read -r op423
461   read -r op424
462   read -r op425
463   read -r op426
464   read -r op427
465   read -r op428
466   read -r op429
467   read -r op430
468   read -r op431
469   read -r op432
470   read -r op433
471   read -r op434
472   read -r op435
473   read -r op436
474   read -r op437
475   read -r op438
476   read -r op439
477   read -r op440
478   read -r op441
479   read -r op442
480   read -r op443
481   read -r op444
482   read -r op445
483   read -r op446
484   read -r op447
485   read -r op448
486   read -r op449
487   read -r op450
488   read -r op451
489   read -r op452
490   read -r op453
491   read -r op454
492   read -r op455
493   read -r op456
494   read -r op457
495   read -r op458
496   read -r op459
497   read -r op460
498   read -r op461
499   read -r op462
500   read -r op463
501   read -r op464
502   read -r op465
503   read -r op466
504   read -r op467
505   read -r op468
506   read -r op469
507   read -r op470
508   read -r op471
509   read -r op472
510   read -r op473
511   read -r op474
512   read -r op475
513   read -r op476
514   read -r op477
515   read -r op478
516   read -r op479
517   read -r op480
518   read -r op481
519   read -r op482
520   read -r op483
521   read -r op484
522   read -r op485
523   read -r op486
524   read -r op487
525   read -r op488
526   read -r op489
527   read -r op490
528   read -r op491
529   read -r op492
530   read -r op493
531   read -r op494
532   read -r op495
533   read -r op496
534   read -r op497
535   read -r op498
536   read -r op499
537   read -r op500
538   read -r op501
539   read -r op502
540   read -r op503
541   read -r op504
542   read -r op505
543   read -r op506
544   read -r op507
545   read -r op508
546   read -r op509
547   read -r op510
548   read -r op511
549   read -r op512
550   read -r op513
551   read -r op514
552   read -r op515
553   read -r op516
554   read -r op517
555   read -r op518
556   read -r op519
557   read -r op520
558   read -r op521
559   read -r op522
560   read -r op523
561   read -r op524
562   read -r op525
563   read -r op526
564   read -r op527
565   read -r op528
566   read -r op529
567   read -r op530
568   read -r op531
569   read -r op532
570   read -r op533
571   read -r op534
572   read -r op535
573   read -r op536
574   read -r op537
575   read -r op538
576   read -r op539
577   read -r op540
578   read -r op541
579   read -r op542
580   read -r op543
581   read -r op544
582   read -r op545
583   read -r op546
584   read -r op547
585   read -r op548
586   read -r op549
587   read -r op550
588   read -r op551
589   read -r op552
590   read -r op553
591   read -r op554
592   read -r op555
593   read -r op556
594   read -r op557
595   read -r op558
596   read -r op559
597   read -r op560
598   read -r op561
599   read -r op562
600   read -r op563
601   read -r op564
602   read -r op565
603   read -r op566
604   read -r op567
605   read -r op568
606   read -r op569
607   read -r op570
608   read -r op571
609   read -r op572
610   read -r op573
611   read -r op574
612   read -r op575
613   read -r op576
614   read -r op577
615   read -r op578
616   read -r op579
617   read -r op580
618   read -r op581
619   read -r op582
620   read -r op583
621   read -r op584
622   read -r op585
623   read -r op586
624   read -r op587
625   read -r op588
626   read -r op589
627   read -r op590
628   read -r op591
629   read -r op592
630   read -r op593
631   read -r op594
632   read -r op595
633   read -r op596
634   read -r op597
635   read -r op598
636   read -r op599
637   read -r op600
638   read -r op601
639   read -r op602
640   read -r op603
641   read -r op604
642   read -r op605
643   read -r op606
644   read -r op607
645   read -r op608
646   read -r op609
647   read -r op610
648   read -r op611
649   read -r op612
650   read -r op613
651   read -r op614
652   read -r op615
653   read -r op616
654   read -r op617
655   read -r op618
656   read -r op619
657   read -r op620
658   read -r op621
659   read -r op622
660   read -r op623
661   read -r op624
662   read -r op625
663   read -r op626
664   read -r op627
665   read -r op628
666   read -r op629
667   read -r op630
668   read -r op631
669   read -r op632
670   read -r op633
671   read -r op634
672   read -r op635
673   read -r op636
674   read -r op637
675   read -r op638
676   read -r op639
677   read -r op640
678   read -r op641
679   read -r op642
680   read -r op643
681   read -r op644
682   read -r op645
683   read -r op646
684   read -r op647
685   read -r op648
686   read -r op649
687   read -r op650
688   read -r op651
689   read -r op652
690   read -r op653
691   read -r op654
692   read -r op655
693   read -r op656
694   read -r op657
695   read -r op658
696   read -r op659
697   read -r op660
698   read -r op661
699   read -r op662
700   read -r op663
701   read -r op664
702   read -r op665
703   read -r op666
704   read -r op667
705   read -r op668
706   read -r op669
707   read -r op670
708   read -r op671
709   read -r op672
710   read -r op673
711   read -r op674
712   read -r op675
713   read -r op676
714   read -r op677
715   read -r op678
716   read -r op679
717   read -r op680
718   read -r op681
719   read -r op682
720   read -r op683
721   read -r op684
722   read -r op685
723   read -r op686
724   read -r op687
725   read -r op688
726   read -r op689
727   read -r op690
728   read -r op691
729   read -r op692
730   read -r op693
731   read -r op694
732   read -r op695
733   read -r op696
734   read -r op697
735   read -r op698
736   read -r op699
737   read -r op700
738   read -r op701
739   read -r op702
740   read -r op703
741   read -r op704
742   read -r op705
743   read -r op706
744   read -r op707
745   read -r op708
746   read -r op709
747   read -r op710
748   read -r op711
749   read -r op712
750   read -r op713
751   read -r op714
752   read -r op715
753   read -r op716
754   read -r op717
755   read -r op718
756   read -r op719
757   read -r op720
758   read -r op721
759   read -r op722
760   read -r op723
761   read -r op724
762   read -r op725
763   read -r op726
764   read -r op727
765   read -r op728
766   read -r op729
767   read -r op730
768   read -r op731
769   read -r op732
770   read -r op733
771   read -r op734
772   read -r op735
773   read -r op736
774   read -r op737
775   read -r op738
776   read -r op739
777   read -r op740
778   read -r op741
779   read -r op742
780   read -r op743
781   read -r op744
782   read -r op745
783   read -r op746
784   read -r op747
785   read -r op748
786   read -r op749
787   read -r op750
788   read -r op751
789   read -r op752
790   read -r op753
791   read -r op754
792   read -r op755
793   read -r op756
794   read -r op757
795   read -r op758
796   read -r op759
797   read -r op760
798   read -r op761
799   read -r op762
800   read -r op763
801   read -r op764
802   read -r op765
803   read -r op766
804   read -r op767
805   read -r op768
806   read -r op769
807   read -r op770
808   read -r op771
809   read -r op772
810   read -r op773
811   read -r op774
812   read -r op775
813   read -r op776
814   read -r op777
815   read -r op778
816   read -r op779
817   read -r op780
818   read -r op781
819   read -r op782
820   read -r op783
821   read -r op784
822   read -r op785
823   read -r op786
824   read -r op787
825   read -r op788
826   read -r op789
827   read -r op790
828   read -r op791
829   read -r op792
830   read -r op793
831   read -r op794
832   read -r op795
833   read -r op796
834   read -r op797
835   read -r op798
836   read -r op799
837   read -r op800
838   read -r op801
839   read -r op802
840   read -r op803
841   read -r op804
842   read -r op805
843   read -r op806
844   read -r op807
845   read -r op808
846   read -r op809
847   read -r op810
848   read -r op811
849   read -r op812
850   read -r op813
851   read -r op814
852   read -r op815
853   read -r op816
854   read -r op817
855   read -r op818
856   read -r op819
857   read -r op820
858   read -r op821
859   read -r op822
860   read -r op823
861   read -r op824
862   read -r op825
863   read -r op826
864   read -r op827
865   read -r op828
866   read -r op829
867   read -r op830
868   read -r op831
869   read -r op832
870   read -r op833
871   read -r op834
872   read -r op835
873   read -r op836
874   read -r op837
875   read -r op838
876   read -r op839
877   read -r op840
878   read -r op841
879   read -r op842
880   read -r op843
881   read -r op844
882   read -r op845
883   read -r op846
884   read -r op847
885   read -r op848
886   read -r op849
887   read -r op850
888   read -r op851
889   read -r op852
890   read -r op853
891   read -r op854
892   read -r op855
893   read -r op856
894   read -r op857
895   read -r op858
896   read -r op859
897   read -r op860
898   read -r op861
899   read -r op862
900   read -r op863
901   read -r op864
902   read -r op865
903   read -r op866
904   read -r op867
905   read -r op868
906   read -r op869
907   read -r op870
908   read -r op871
909   read -r op872
910   read -r op873
911   read -r op874
912   read -r op875
913   read -r op876
914   read -r op877
915   read -r op878
916   read -r op879
917   read -r op880
918   read -r op881
919   read -r op882
920   read -r op883
921   read -r op884
922   read -r op885
923   read -r op886
924   read -r op887
925   read -r op888
926   read -r op889
927   read -r op890
928   read -r op891
929   read -r op892
930   read -r op893
931   read -r op894
932   read -r op895
933   read -r op896
934   read -r op897
935   read -r op898
936   read -r op899
937   read -r op900
938   read -r op901
939   read -r op902
940   read -r op903
941   read -r op904
942   read -r op905
943   read -r op906
944   read -r op907
945   read -r op908
946   read -r op909
947   read -r op910
948   read -r op911
949   read -r op912
950   read -r op913
951   read -r op914
952   read -r op915
953   read -r op916
954   read -r op917
955   read -r op918
956   read -r op919
957   read -r op920
958   read -r op921
959   read -r op922
960   read -r op923
961   read -r op924
962   read -r op925
963   read -r op926
964   read -r op927
965   read -r op928
966   read -r op929
967   read -r op930
968   read -r op931
969   read -r op932
970   read -r op933
971   read -r op934
972   read -r op935
973   read -r op936
974   read -r op937
975   read -r op938
976   read -r op939
977   read -r op940
978   read -r op941
979   read -r op942
980   read -r op943
981   read -r op944
982   read -r op945
983   read -r op946
984   read -r op947
985   read -r op948
986   read -r op949
987   read -r op950
988   read -r op951
989   read -r op952
990   read -r op953
991   read -r op954
992   read -r op955
993   read -r op956
994   read -r op957
995   read -r op958
996   read -r op959
997   read -r op960
998   read -r op961
999   read -r op962
1000  read -r op963
1001  read -r op964
1002  read -r op965
1003  read -r op966
100
```

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Все получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же

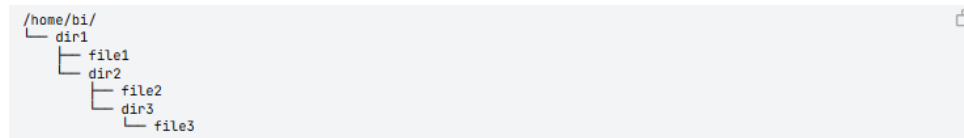
Следующий шаг

Решить снова

Задание 22

Выполнение 23 задания (рис. 23).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:



Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

✓ Верно.

- ☐ Только `file1`
- ☐ Только `file2`
- ☒ Все кроме `file3`
- ☐ Все кроме `file1`
- ☐ Все кроме `file2`

Следующий шаг

Решить снова

Задание 23

Выполнение 24 задания (рис. 24).

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем **в каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

✓ Отличное решение!

- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`

Следующий шаг

Решить снова

Задание 24

Выполнение 25 задания (рис. 25).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☒ The best OS is Xubuntu
- ☒ I prefer Kubuntu
- ☒ Hmm, XKLubuntu
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu

Следующий шаг

Решить снова

Задание 25

Выполнение 26 задания (рис. 26).

Что произойдет, если в команде `sed -n "[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

✓ Отличное решение!

- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☒ Каждая строчка будет выведена два раза
- ☐ Появится сообщение об ошибке

Следующий шаг

Решить снова

Задание 26

Выполнение 27 задания (рис. 27).

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции **должны быть** указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст `" YOU YOU and YOU!"` является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLA and DM !"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation !"`.

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" **количество пробелов** в тексте **не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск **не идеально повторяет** `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Прекрасный ответ.

```
sed -r 's/[A-Z]{2,} / abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Задание 27

Выполнение 28 задания (рис. 28).

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Правильно.

- ☒ `-p, -persist`
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ `-raise`
- ☐ Такой опции не существует

Следующий шаг

Решить снова

Задание 28

Выполнение 29 задания (рис. 29).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет название у построенного ряда данных и сколько будет нарисовано точек на графике?

Выберите один вариант из списка

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название "data.csv using 1:2", нарисовано 10 точек
- ☐ Название "no name", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек

Следующий шаг

Решить снова

Задание 29

Выполнение 30 задания (рис. 30).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде "point <номер точки>, value <значение соответствующей переменной>". Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под конкатенацией обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Правильно, молодец!

```
set xtics ("point 1, value "x1 x1 , "point 2, value "x2 x2 , "point 3, value "x3 x3)
```

Следующий шаг

Решить снова

Задание 30

Выполнение 31 задания (рис. 31).

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Верно.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Задание 31

Выполнение 32 задания (рис. 32).

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные варианты ответа!**

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнялись все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☐ `chmod 777 file.txt`
- ☐ `chmod rwxrw-r-- file.txt`
- ☒ `chmod 764 file.txt`

Следующий шаг

Решить снова

Задание 32

Выполнение 33 задания (рис. 33).

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rwXr-x` (владелец root, группа root). Таким образом никто кроме пользователя root не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды user из группы group всё-таки сможет создать файл внутри `dir`? Укажите **все верные варианты ответа!**

Примечание: считаем, что все команды выполняются от имени user, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно. Дело в том, что если создать при помощи `sudo` файл с правами `rwXr-x` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять все файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для root). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rwXr-x`, пользователь может сделать с этим файлом почти всё что угодно! В случае же, когда речь идет о директории созданной root, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет). Важно отметить, что директории в Linux это в каком-то смысле файлы. Содержимое такого "файла" -- это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней. На самом деле и это еще не всё. Существует так называемый **sticky bit** (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса Alexey Antipovskiy за помощь в оформлении Примечания 2!

Выберите все подходящие ответы из списка

☒ Точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chown user dir`
- ☒ `sudo chmod a+w dir`
- ☒ `sudo chown user:group dir`
- ☐ `sudo chmod g+w dir`
- ☐ `sudo chmod o+X dir`
- ☒ `sudo chmod o+w dir`

Следующий шаг

Решить снова

Задание 33

Выполнение 34 задания (рис. 34).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✔ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✔ Количество символов
- ✔ Количество слов
- ✔ Количество строк
- ✔ Длину самой длинной строки
- ✔ Размер файла в байтах

Следующий шаг

Решить снова

Задание 34

Выполнение 35 задания (рис. 35).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо `2848 байт` надо вывести `2.8K`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4M` (также на экране может быть выведен еще и символ `" "`, обозначающий, что это размер именно текущей директории).

Напишите текст

✔ Здорово, всё верно.

```
du -h -s
```

Следующий шаг

Решить снова

Задание 35

Выводы

Успешно выполнена 1 часть курса на stepic